



UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 4 – Somadores e ULA de 4 Bits

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Setembro de
2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 4 – Somadores e ULA de 4 Bits

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1 – SOMADORES RIPPLE-CARRY

1.1 Item (a) – Somador completo de 1 bit (estrutural)

O somador completo foi implementado de forma estrutural, instanciando apenas portas lógicas básicas (`and2`, `or2`, `xor2`), sem nenhuma atribuição booleana direta. A soma é o XOR encadeado de `a`, `b` e `cin`; o carry-out é o OR entre o AND de `a` e `b` e o AND do carry-in com o XOR parcial de `a` e `b`.

```

1
2 — Primeira porta l gica: AND
3 library ieee;
4 use ieee.std_logic_1164.all;
5
6 entity gate_and2 is
7     port (
8         a, b : in  std_logic;
9         y    : out std_logic
10    );
11 end entity gate_and2;
12
13 architecture rtl of gate_and2 is
14 begin
15     y <= a and b;
16 end architecture rtl;
17
18
19 — Segunda porta l gica: OR
20 library ieee;
21 use ieee.std_logic_1164.all;
22
23 entity gate_or2 is
24     port (
25         a, b : in  std_logic;
26         y    : out std_logic
27    );
28 end entity gate_or2;
29

```

```

30 architecture rtl of gate_or2 is
31 begin
32     y <= a or b;
33 end architecture rtl;
34
35 — Terceira porta l gica: XOR
36 library ieee;
37 use ieee.std_logic_1164.all;
38
39 entity gate_xor2 is
40     port (
41         a, b : in  std_logic;
42         y     : out std_logic
43     );
44 end entity gate_xor2;
45
46 architecture rtl of gate_xor2 is
47 begin
48     y <= a xor b;
49 end architecture rtl;

```

Abaixo, um arquivo separado, intitulado `full_adder.vhdl`, instancia as portas lógicas e mapeia as operações para a soma de 1 bit. Ao todo são usados 3 bits de entrada (`a`, `b` e `cin`), resultando em uma saída de 2 bits (`s` e `cout`).

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity full_adder is
5     port (
6         a, b, cin : in  std_logic;
7         s, cout   : out std_logic
8     );
9 end entity full_adder;
10
11 architecture structural of full_adder is
12     signal a_xor_b    : std_logic;
13     signal a_and_b    : std_logic;
14     signal cin_and_x  : std_logic;
15 begin

```

```

16 xor_ab: entity work.gate_xor2
17     port map (a => a, b => b, y => a_xor_b);
18
19 and_ab: entity work.gate_and2
20     port map (a => a, b => b, y => a_and_b);
21
22 xor_final: entity work.gate_xor2
23     port map (a => a_xor_b, b => cin, y => s);
24
25 and_cin: entity work.gate_and2
26     port map (a => cin, b => a_xor_b, y => cin_and_x);
27
28 or_carry: entity work.gate_or2
29     port map (a => a_and_b, b => cin_and_x, y => cout);
30
31 end architecture structural;

```

1.2 Item (b) – Simulação do somador completo

A simulação aplica sequencialmente as 8 combinações possíveis de **a**, **b** e **cin**, com 10 ns entre cada estímulo. Ela foi executada via script **sim.do**, gerando o arquivo de onda **full_adder.vcd**.

Arquivo **tb_full_adder.vhdl**, usado para a simulação:

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity tb_full_adder is
5 end entity tb_full_adder;
6
7 architecture sim of tb_full_adder is
8     signal a, b, cin : std_logic;
9     signal s, cout   : std_logic;
10 begin
11     uut: entity work.full_adder
12         port map (
13             a    => a,
14             b    => b,
15             cin  => cin,

```

```

16         s      => s ,
17         cout => cout
18     );
19
20     stim: process
21     begin
22         a <= '0'; b <= '0'; cin <= '0'; wait for 10 ns; — 000
23         a <= '0'; b <= '0'; cin <= '1'; wait for 10 ns; — 001
24         a <= '0'; b <= '1'; cin <= '0'; wait for 10 ns; — 010
25         a <= '0'; b <= '1'; cin <= '1'; wait for 10 ns; — 011
26         a <= '1'; b <= '0'; cin <= '0'; wait for 10 ns; — 100
27         a <= '1'; b <= '0'; cin <= '1'; wait for 10 ns; — 101
28         a <= '1'; b <= '1'; cin <= '0'; wait for 10 ns; — 110
29         a <= '1'; b <= '1'; cin <= '1'; wait for 10 ns; — 111
30         wait;
31     end process;
32
33 end architecture sim;

```

1.3 Item (c) – Ripple-carry adder de 4 bits

O somador de 4 bits encadeia quatro instâncias do `full_adder` do item (a) usando `generate`: o carry-out de cada estágio alimenta o carry-in do próximo. Os operandos `x` e `y` são tratados como números sem sinal de 4 bits, de modo que o overflow é exatamente o carry-out do último estágio (`carry(4)`).

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity ripple_carry_adder4 is
5      port (
6          x, y      : in  std_logic_vector(3 downto 0);
7          cin       : in  std_logic;
8          sum       : out std_logic_vector(3 downto 0);
9          overflow  : out std_logic
10     );
11 end entity ripple_carry_adder4;
12
13 architecture structural of ripple_carry_adder4 is

```

```

14     signal carry : std_logic_vector(4 downto 0);
15 begin
16     carry(0) <= cin;
17
18     fa_gen: for i in 0 to 3 generate
19         fa_inst: entity work.full_adder
20             port map (
21                 a    => x(i),
22                 b    => y(i),
23                 cin  => carry(i),
24                 s    => sum(i),
25                 cout => carry(i + 1)
26             );
27     end generate fa_gen;
28
29     overflow <= carry(4);
30
31 end architecture structural;

```

1.4 Item (d) – Instanciação no demo_setup

O somador ripple_carry_adder4 foi instanciado no demo_setup. Os 8 *toggle switches* são divididos em duas entradas de 4 bits (SW(3 downto 0) = x, SW(7 downto 4) = y), a soma é decodificada no display hexadecimal e o overflow acende um LED.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity demo_setup is
5      port (
6          SW      : in  std_logic_vector(7 downto 0);
7          HEX3    : out std_logic_vector(6  downto 0);
8          LEDR0   : out std_logic
9      );
10 end entity demo_setup;
11
12 architecture rtl of demo_setup is
13     signal sum_result : std_logic_vector(3 downto 0);
14     signal overflow_bit : std_logic;

```

```

15 begin
16     adder_inst: entity work.ripple_carry_adder4
17         port map (
18             x          => SW(3 downto 0),
19             y          => SW(7 downto 4),
20             cin        => '0',
21             sum         => sum_result,
22             overflow    => overflow_bit
23         );
24
25     seg_inst: entity work.conv_7seg
26         port map (
27             HEX_IN      => sum_result,
28             HEX_OUT     => HEX3
29         );
30
31     LEDR0 <= overflow_bit;
32
33 end architecture rtl;

```

1.5 Itens (e), (f) e (g) – Ripple-carry de 8, 32 e 64 bits

Os somadores de 8, 32 e 64 bits seguem exatamente a mesma estrutura do item (c), apenas variando a largura dos vetores e o número de `full_adder` encadeados no `generate`. O overflow continua sendo o carry-out do último estágio.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity adder8 is
5     port (
6         x, y      : in  std_logic_vector(7 downto 0);
7         cin       : in  std_logic;
8         sum       : out std_logic_vector(7 downto 0);
9         overflow  : out std_logic
10    );
11 end entity adder8;
12
13 architecture structural of adder8 is

```

```

14     signal carry : std_logic_vector(8 downto 0);
15 begin
16     carry(0) <= cin;
17
18     fa_gen: for i in 0 to 7 generate
19         fa_inst: entity work.full_adder
20             port map (
21                 a    => x(i),
22                 b    => y(i),
23                 cin  => carry(i),
24                 s    => sum(i),
25                 cout => carry(i + 1)
26             );
27     end generate fa_gen;
28
29     overflow <= carry(8);
30
31 end architecture structural;

```

Os somadores de 32 e 64 bits (`adder32.vhdl` e `adder64.vhdl`) têm arquitetura idêntica à do `adder8`, apenas com os vetores estendidos para 32 e 64 bits, respectivamente, e o `generate` percorrendo 0 to 31 ou 0 to 63.

Cada somador foi compilado em um projeto Quartus próprio (`compile_adder8.tcl`, `compile_adder32.tcl`, `compile_adder64.tcl`, todos para a família Cyclone II, dispositivo EP2C20F484C7), e o atraso combinacional foi medido com o TimeQuest Timing Analyzer via `measure_delay.tcl`, que gera o relatório de *Propagation Delay* (`report_datasheet`) para cada projeto já compilado.

1.6 Item (h)

Nao sei

1.7 Item (i) – Relação entre atraso e tamanho da entrada

Os tempos de atraso combinacional foram medidos dentro do container Quartus (Cyclone II EP2C20F484C7, Quartus II 13.0sp1), compilando cada projeto e em seguida rodando o TimeQuest (`report_datasheet`). O atraso do circuito inteiro é o maior valor

da tabela de *Propagation Delay*, ou seja, o pior caminho combinacional entre uma entrada e uma saída:

Somador	Atraso medido	Pior caminho
8 bits	19,860 ns	x[0] $\rightarrow$ overflow
32 bits	42,970 ns	cin $\rightarrow$ overflow
64 bits	71,831 ns	y[0] $\rightarrow$ overflow

O atraso cresce com N , porém de forma sub-linear: de 8 para 32 bits ($4\times$ mais bits) o atraso sobe apenas $\sim 2,16\times$; de 32 para 64 bits ($2\times$ mais bits) sobe apenas $\sim 1,67\times$. Isso acontece porque o caminho crítico é sempre a cadeia de carries até o *overflow*, e cada *full adder* adicionado só aumenta esse caminho em um estágio.

2 QUESTÃO 2 – ULA DE 4 BITS

2.1 Item (a) – Display de números negativos

Para mostrar números negativos de 4 bits (complemento de 2) em decimal, o circuito primeiro verifica o bit de sinal. Se for negativo, o valor é complementado (invertido e somado a 1) para obter a magnitude, e um LED indica o sinal; caso contrário, a magnitude é o próprio valor. A magnitude (0 a 8) é então decodificada para o display de 7 segmentos, do mesmo jeito que nos displays decimais anteriores.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity display_signed is
6      port (
7          VALUE_IN : in  std_logic_vector(3 downto 0);
8          HEX_OUT  : out std_logic_vector(6 downto 0);
9          SIGN_LED : out std_logic
10     );
11 end entity display_signed;
12
13 architecture rtl of display_signed is
14     signal magnitude : unsigned(3 downto 0);
15 begin
16     process (VALUE_IN)
17     begin
18         if VALUE_IN(3) = '1' then
19             magnitude <= unsigned(not VALUE_IN) + 1;
20             SIGN_LED  <= '1';
21         else
22             magnitude <= unsigned(VALUE_IN);
23             SIGN_LED  <= '0';
24         end if;
25     end process;
26
27     with std_logic_vector(magnitude) select

```

```
28     HEX_OUT <= "1000000" when "0000", — 0
29               "1111001" when "0001", — 1
30               "0100100" when "0010", — 2
31               "0110000" when "0011", — 3
32               "0011001" when "0100", — 4
33               "0010010" when "0101", — 5
34               "0000010" when "0110", — 6
35               "1111000" when "0111", — 7
36               "0000000" when "1000", — 8
37               "1111111" when others;
38
39 end architecture rtl;
```